

Campus Recruitment Training Phase 2

Name : Mohammad Shayan Qureshi

Sem : 6

Batch - B1

Roll No: B2-33

Q1)

Output

Constructor called

Init called

Q2)

Corrected code:

return new ResponseEntity<>(u, [HttpStatus.OK](#));

Q3)

The image shows two side-by-side screenshots. The left screenshot is a Postman interface for a POST request to `http://localhost:8080/api/students`. The request body is a JSON object: `{ "name": "Shayan", "age": 20 }`. The response body shows the message `Student created successfully`. The right screenshot shows a JSON response with a 400 status code and an error message: `{ "timestamp": "2026-05-27T10:00:49.385Z", "status": 400, "error": "Bad Request", "path": "/api/students" }`.

Q4)

Annotation : @Transactional

Both credit and debit are treated as single operation , either they execute successfully or fail , there is no inconsistent state.

Q6)

Corrected code:

String token = header.substring(7);

Filter validates jwt request header

Q8)

Problem: N+1 query problem occurs when

1 query loads parent entities and N queries load related child entities.

Cause: Lazy loading inside loops.

Fix: Use JOIN FETCH in JPQL to load relationships in one query.

Q11)

Fixed code:

@Query("SELECT o FROM Order o JOIN FETCH o.items")

Q16)

Benefits of splitting into microservices:

1. Each module or service can be deployed and scaled separately
2. Crash of one service doesn't affect others
3. We can use separate tech stack for each service and improve its performance

Challenges

1. Increased complexity
2. Harder debugging and monitoring because logs and flows are distributed across multiple services.

(c) Eureka

Eureka acts as a **service registry** where all microservices (User, Order, Product) register themselves with their service names

The API Gateway acts as a **single entry point** for the React frontend, which forwards incoming requests to the correct service by looking up their locations in Eureka dynamically.